

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Analytical Problem Solving: 40%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I K. Kabilan (192521157) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K. Kabilan

1. RSA Encryption Analysis.

Given:

$$* p = 11$$

$$* q = 13$$

$$* e = 7$$

$$* M = 9$$

a) compute n and $\phi(n)$

$$\begin{aligned} n &= p \times q = 11 \times 13 \\ &= 143 \end{aligned}$$

$$\begin{aligned} \phi(n) &= (11-1)(13-1) \\ &= 10 \times 12 \\ &= 120 \end{aligned}$$

$$* n = 143$$

$$* \phi(n) = 120$$

b) Find private key d

Find d such that

$$7d \equiv 1 \pmod{120}$$

using the Extended Euclidean Algorithm,

$$7 \times 103 = 721$$

$$721 \text{ mod } 120 = 1$$

Therefore

private key

$$d = 103$$

Encrypt M=9

Formula:

$$C = M^e \bmod n$$

$$C = 9^7 \bmod 143$$

using modular arithmetic,

$$9^7 = 4782969$$

$$4782969 \bmod 143 = 48$$

ciphertext

$$\boxed{C=48}$$

d) Decrypt ciphertext

Formula

$$M = C^d \bmod n$$

$$M = 48^{103} \bmod 143$$

using modular exponentiation,

$$M = 9$$

verified

original message = 9

e) Effect of changing e

* Small $e \rightarrow$ Faster encryption but may introduce vulnerabilities if used improperly.

* Large $e \rightarrow$ slower encryption but increase resistance to certain attacks.

* Common secure value is 65537, which balances security and efficiency.

2. Block cipher Mode Evaluation

Given

plaintext

$$P = [1010, 1010, 1111, 1010]$$

Key

$$K = 1100$$

$$IV = 0000$$

Assume encryption is XOR with the Key.

a) ECB Mode (Electronic code book)

Encryption:

$$C = P \oplus K$$

Plaintext	Key	ciphertext
1010	1100	0110
1010	1100	0110
1111	1100	0011
1010	1100	0110

ECB output

$$[0110, 0110, 0011, 0110]$$

Encrypt M=9

CBC Mode Cipher block chaining

Block 1

$$000 \text{ XOR } 1010 = 1010$$

$$1010 \text{ XOR } 1100 = 0110$$

$$C1 = 0110$$

Block 2

$$1010 \text{ XOR } 0110 = 1100$$

$$1100 \text{ XOR } 1100 = 0000$$

$$C2 = 0000$$

Block 3

$$1111 \text{ XOR } 0000 = 1111$$

$$1111 \text{ XOR } 1100 = 0011$$

$$C3 = 0011$$

Block 4

$$1010 \text{ XOR } 0011 = 1001$$

$$1001 \text{ XOR } 1100 = 0101$$

$$C4 = 0101$$

CBC output

[0110, 0000, 0011, 0101]

b) Comparison

ECB

0110 0110 0111 0110

CBC

0110 0000 0011 0101

Repeated plaintext produces repeated ciphertext only in ECB

c) which is better?

CBC is better each plaintext block is XORed with the previous ciphertext block before encryption. This hides repeated patterns and provides stronger security than ECB.

3. DES VS 3-DES VS Blowfish (Data Encryption Standard)

Given

DES = 2 ms / block

3DES = 6 ms / block

Blowfish = 1 ms / block

1000 blocks

i) Total Encryption Time

DES

$$1000 \times 2 = 2000 \text{ ms} = 2 \text{ Seconds}$$

3DES

$$1000 \times 6 = 6000 \text{ ms} = 6 \text{ seconds}$$

Blowfish

$$1000 \times 1 = 1000 \text{ ms} = 1 \text{ second}$$

Algorithm	Key Size	Time
DES	56 bits	2 sec
3DES	168 bits	6 sec
Blowfish	128 bits	1 sec

ii) Trade-off

* DES: Fast but increase due to its small 56-bit key.

* 3DES: Stronger security but significantly slower.

* Blowfish: High security with the fastest performance among the three.

iii) Recommendation

Blowfish

Reason:

* Fastest

* Secure 128-bit key

* Suitable for real-time communication

4. Diffie - Hellman Key Exchange

Primen

$$P = 23$$

$$g = 5$$

private Key $A = 6$

private Key $B = 15$

d) public keys

user A

$$A = 5^6 \bmod 23$$

$$5^6 = 15625$$

$$15625 \bmod 23 = 8$$

public Key of $A = 8$

user B

$$B = 5^{15} \bmod 23$$

Result

$$B = 19$$

public Key of $B = 19$

e) shared secret

user A

$$19^6 \bmod 23 = 2$$

user B

$$8^{15} \bmod 23 = 2$$

shared secret

2

3) Man-in-the-middle Attack

An attacker intercepts the exchanged public keys and replaces with their own public keys. User A and user B each establish a shared key with the attacker instead of with each other, allowing the attacker to decrypt, modify, and re-encrypt messages without either user noticing.

g) Prevention

- * use Digital signature
- * use Digital Certificate (PKI)
- * Authenticate both used before exchanging keys.

5. ECC vs RSA (Elliptic curve cryptography vs Rivest Shamir and Adleman).

Given

RSA = 1024 bits

ECC = 160 bits

IoT Device

d) Computational cost

RSA

- * Larger Keys
- * Slower computations

ECC

- * Smaller Keys
- * Faster computations

e) Power and memory

ECC

- * Lower power consumption
- * Less memory usage
- * better battery life

RSA

- * Higher memory usage
- * Higher processing power required

f) which is better?

ECC is the better choice for IoT devices because it provides security equivalent to a 1024-bit RSA key while using only a 160-bit key. This results in faster computation, lower memory usage, and reduced power consumption, making it ideal for devices with limited processing capacity.

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	K. Kabilan	Reg. No.:	192521157
Section / Batch:		Date:	20/07/2026

Code of Conduct

I, K. Kabilan (192521157) _____ (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:		100 Marks	

13	192521157	KABILANK	1. How a queue may be maintained by a circular array with $\text{MAXSIZE} = 6$. Observe that queue always occupies consecutive locations except when it occupies locations at the beginning and at the end of the array. If the queue is viewed as a circular array, this means that it still occupies consecutive locations. The queue is empty when $\text{count} = 0$; Perform the following Enqueue and Dequeue operations. a) Initially QUEUE is empty, b) A, B, C are Enqueued, c) A is Dequeued, d) D, E, F are Enqueued, e) B & C are Dequeued, f) G is Enqueued, g) D & E are Dequeued, h) H & I are Enqueued, i) F is dequeued, j) G & H are dequeued k) I is Dequeued. Demonstrate with a neat sketch of Front, Rear and count in a queue and circular queue. 2. Show the linked list configuration resulting from each series of list operations using the List ADT. Assume that lists L1 and L2 are empty at the beginning of each series. Show where the current position is in the list (a) L1.append(10); L1.append(20); L1.append(15); (b) L2.append(10); L2.append(20); L2.append(15); L2.moveToStart(); L2.insert(39); L2.next(); L2.insert(12);
----	-----------	----------	---

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge

Signature: _____

Date: _____

Course Coordinator

Signature: _____

Date: _____

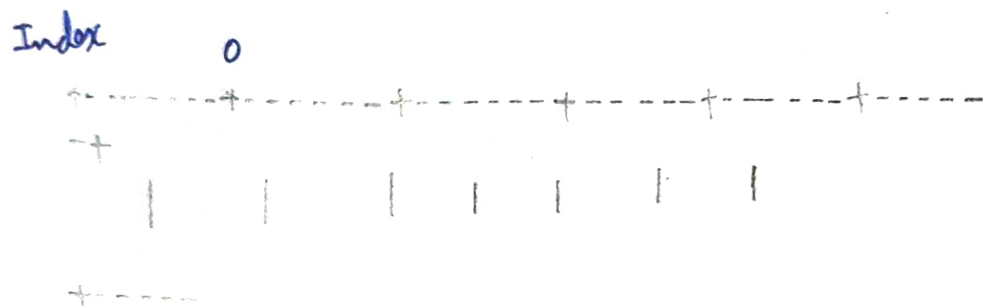
Program Director

Signature: _____

Date: _____

1. Circular Queue using Array - $\text{MAX SIZE} = 6$

A circular queue uses an array of size 6:



we use:

↳ FRONT → points to the first element.

↳ REAR → points to the last element.

↳ COUNT → number of elements currently in the queue.

↳ Initially: $\text{FRONT} = 0$, $\text{REAR} = -1$, $\text{COUNT} = 0$.

↳ Circular movement:

$$\text{REAR} = (\text{REAR} + 1) \bmod 6$$

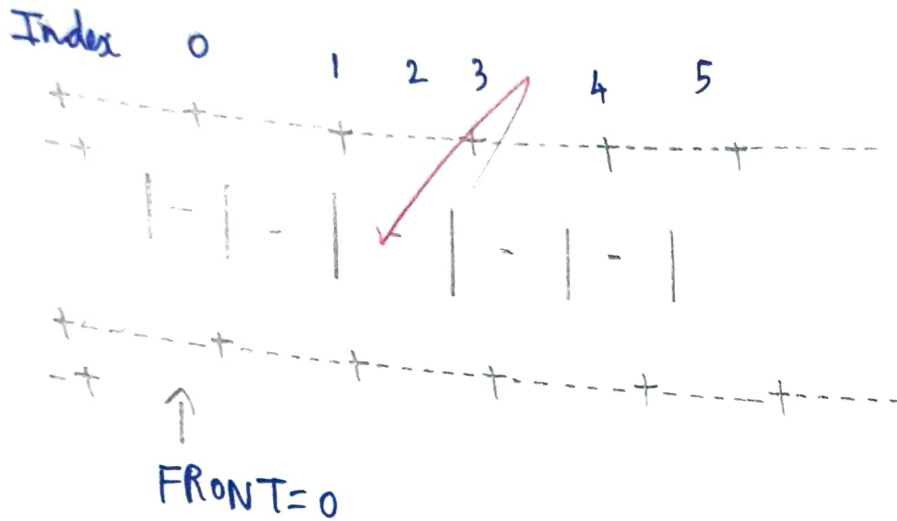
$$\text{FRONT} = (\text{FRONT} + 1) \bmod 6$$

COUNT=0, the queue is empty.

When the rear reaches index 5 and another element is added, it wraps around to index 0.

Steps:

a) Initially QUEUE is empty



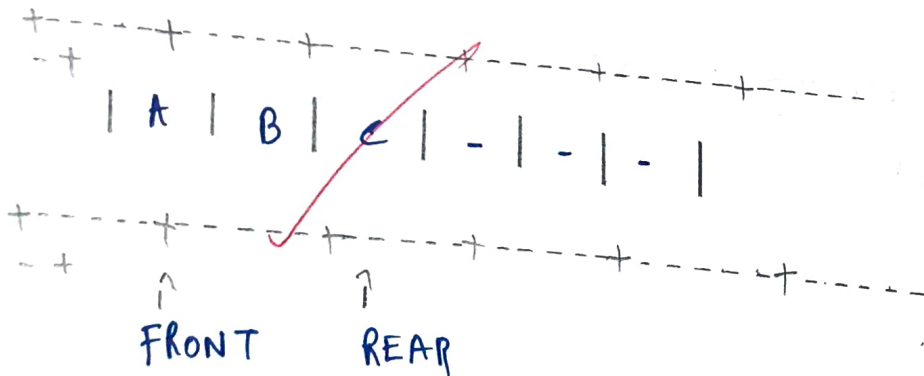
FRONT=0

REAR=-1

COUNT=0

b) A, B, C are Enqueued

After Enqueue (A), Enqueue (B), Enqueue (C):



FRONT=0

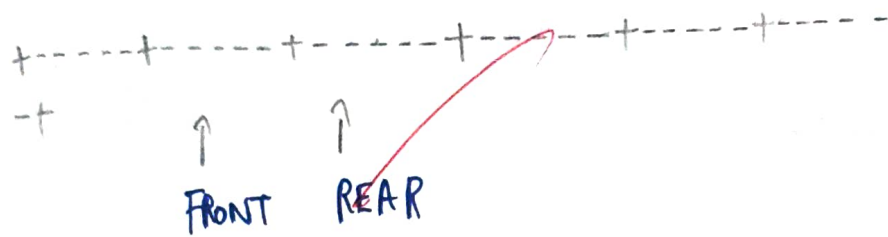
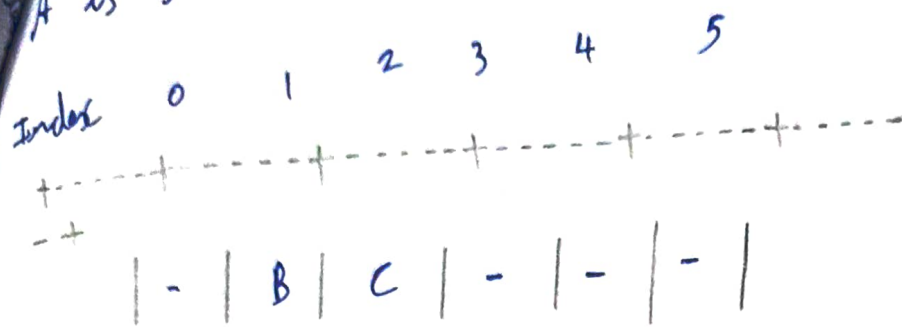
REAR=2

COUNT=3

Queue:

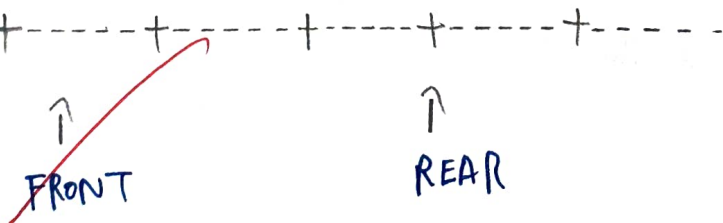
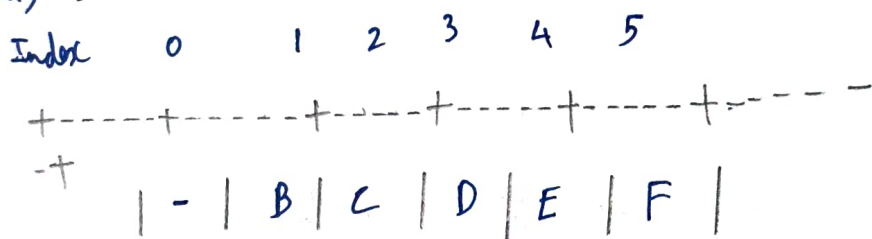
FRONT $\rightarrow$ A $\rightarrow$ B $\rightarrow$ C $\leftarrow$ REAR

A is Dequeued
A is removed. FRONT moves from 0 to 1.



FRONT = 1
REAR = 2
COUNT = 2

d) D, E, F are Enqueued

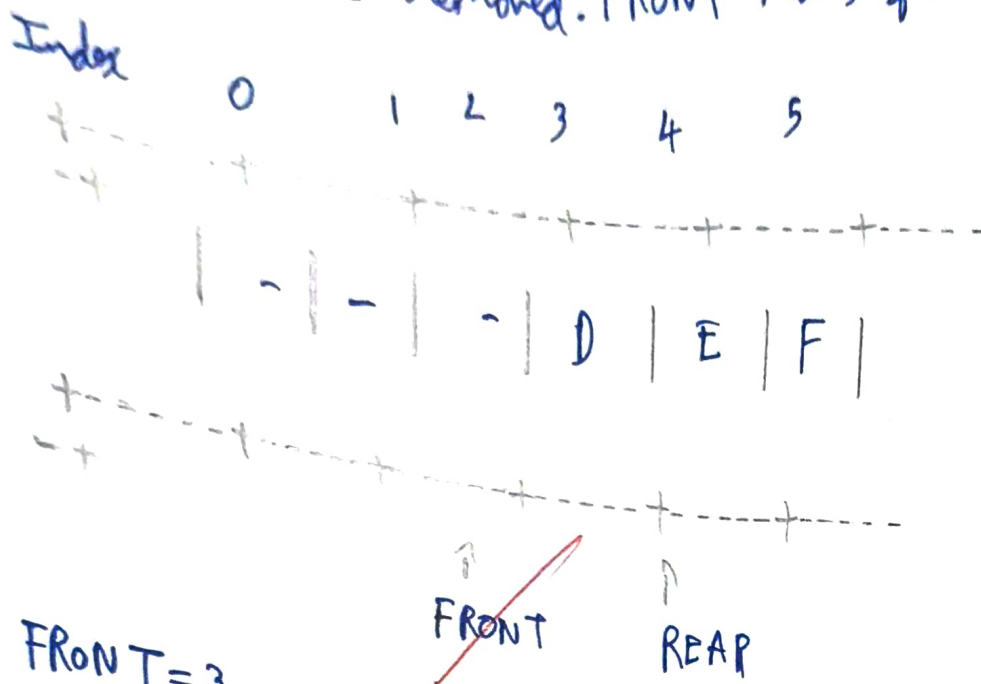


FRONT = 1
REAR = 5
COUNT = 5

Queue:

FRONT $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D $\rightarrow$ E $\rightarrow$ F $\leftarrow$ REAR

B & C are Dequeued
B and C are removed. FRONT moves from 1 $\rightarrow$ 2 $\rightarrow$ 3.



FRONT = 3

REAR = 5

COUNT = 3

Queue:

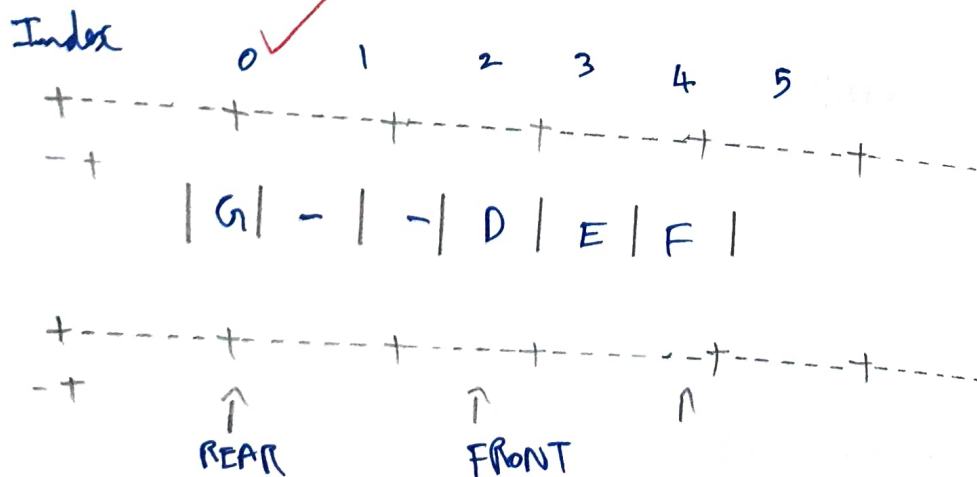
FRONT $\rightarrow$ D $\rightarrow$ E $\rightarrow$ F $\leftarrow$ REAR

f) G is Enqueued

REAR is currently 5. Since the queue is circular:

$$(5+1) \bmod 6 = 0$$

So G is inserted at index 0.



ally, when showing the logical queue, FRONT is at index 0.
REAR is at index 0:

FRONT $\rightarrow$ D $\rightarrow$ E $\rightarrow$ F $\rightarrow$ G $\leftarrow$ REAR
(wraps around)

therefore:

FRONT=3

REAR = 0

COUNT=4

g) D & E are ~~Required~~

p and E are removed

p and E are

	0	1	2	3	4	5
Index						

Index 0 1

+-----+-----+-----+-----+

- +

|G| - |I| - |F|

REAR

FRONT

FRONT = 3

REAR = 0

~~COUNT=2~~

Logical queue;

FRONT $\rightarrow$ F $\rightarrow$ GL $\rightarrow$ REAR

h) H & I are Enqueued

H goes into index 1 and I goes into index 2.

Index 0 1 2 3 4 5

| G | H | I | - | - | F |

Logical circular order: REAR FRONT

FRONT $\rightarrow$ F $\rightarrow$ G $\rightarrow$ H $\rightarrow$ I $\leftarrow$ REAR

Therefore:

FRONT = 5
REAR = 2
COUNT = 4

i) F is Dequeued

F is at index 5. FRONT moves circularly to 0.

Index 0 1 2 3 4 5

| G | H | I | - | - | - |

FRONT REAR

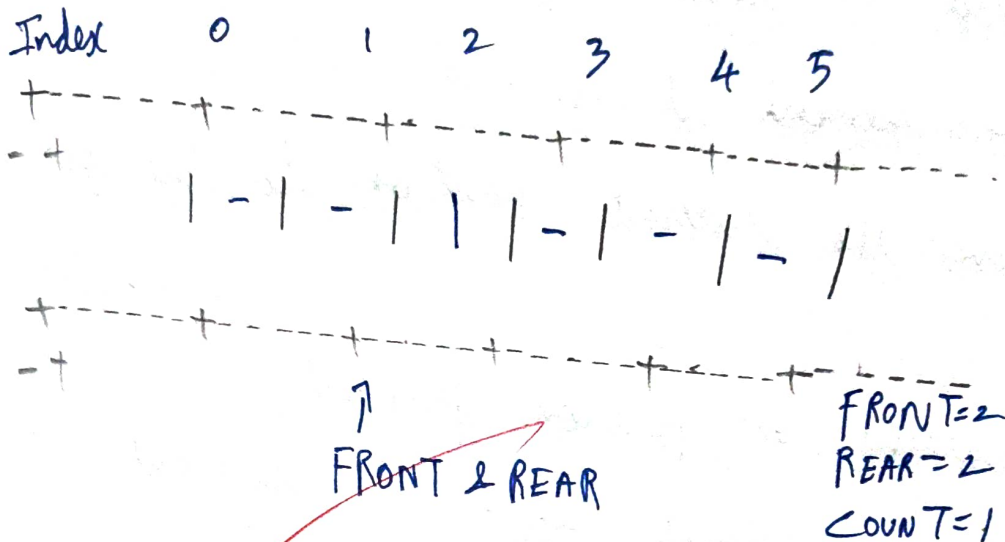
FRONT = 0
REAR = 2
COUNT = 3

here:

FRONT $\rightarrow$ G $\rightarrow$ H $\rightarrow$ I $\leftarrow$ REAR

j) G & H are Dequeued

G and H are removed

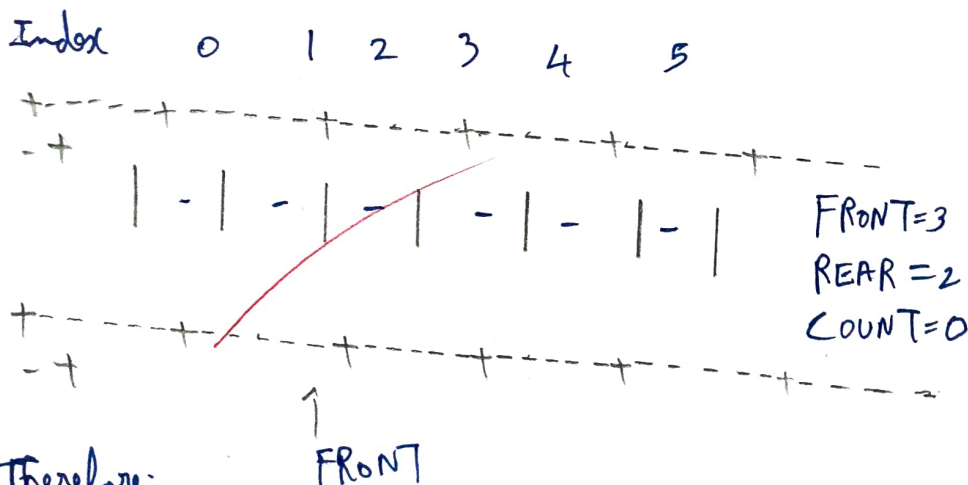


Queue:

FRONT $\rightarrow$ I $\leftarrow$ REAR

k) I is Dequeued

I is removed.



Therefore:

QUEUE is EMPTY

COUNT=0

2. Linked List Configuration using List APT

Assume:

- L₁ and L₂ are initially empty.
- The current position is shown by CUR
- append(x) adds an element at the end.
- moveToStart() moves the current position to the first element.
- next() moves current one position forward.
- insert(x) inserts an element before the current position.

a)
operations:

L₁.append(10);
L₁.append(20);
L₁.append(15)

Step 1

L₁.append(10)

HEAD
↓
[10 | NULL]
↑
CUR

Step 2

L₁.append(20)

AD
101.} → [20 | NULL]
↑
COR

step 3

L1.append(15)

Final linked list:

HEAD
↓
[10 |] → [20 |] → [15 | NULL]
↑
COR

Therefore:

L1 = 10 → 20 → 15 → NULL

Current position = 10

b)

operations:

L2.append(10);

L2.append(20);

L2.append(15);

L2.moveToStart();

L2.insert(39);

L2.reset();

L2.insert(12);

Step 1: Append 10, 20, 15

HEAD

↓

[10 | .] → [20 | .] → [15 | NULL]

Initially, after the appends, the current position

Step 2: L2.moveToStart()

HEAD

↓

[10 | .] → [20 | .] → [15 | NULL]

↑

CUR

Step 3: L2.insert(39)

HEAD

↓

[39 | .] → [10 | .] → [20 | .] → [15 | NULL]

↑

CUR

Step 4: L2.next()

HEAD

↓

[39 | .] → [10 | .] → [20 | .] → [15 | NULL]

↑

CUR

Step 5: L2.insert(12)

HEAD

↓

[39 | .] → [12 | .] → [10 | .] → [20 | .] → [15 | NULL]

↑

CUR